APCS 程式設計系列 · U12 · 實作高級 · C++版

圖論與樹 重點講義

鄰接矩陣與鄰接串列 · BFS 最短步數 · DFS 與連通塊 · 樹的深度與子樹 · 前中後序 · 樹直徑 · 拓撲排序

1. 圖的表示：鄰接矩陣 vs 鄰接串列

圖由「點」和「邊」組成。題目通常給 n 個點、m 條邊，每條邊一行「u v」。第一步永遠是把邊存起來，存法有兩種：

比較	鄰接矩陣 a[u][v]	鄰接串列 adj[u]
空間	n^2 格	n + 2m 個（無向圖每條邊存兩次）
查 u、v 有沒有相連	O(1)：看 a[u][v]	要掃過 u 的所有鄰居
走訪 u 的所有鄰居	O(n)：整列掃一遍	O(u 的度數)：只看真的鄰居
適合	n 不超過約 2000	n 到 10^5 以上（APCS 高級主流）

C++ 建鄰接串列（無向圖每條邊加兩次）

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);    // 點編號 1~n，開 n+1 格
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);           // 無向圖：反方向也要加
    }
    for (int u = 1; u <= n; u++) {
        cout << u << ":";
        for (int v : adj[u]) cout << " " << v;
        cout << "\n";
    }
}
```

輸入	輸出
4 4 1 2 1 3 2 3 3 4	1: 2 3 2: 1 3 3: 1 2 4 4: 3

陷阱提醒 存圖的三個經典錯誤

【錯】無向圖只加一次邊：從 v 走不回 u，BFS / DFS 會漏掉一大片點。

【錯】點編號從 1 開始，卻只開 n 格：存到 adj[n] 就越界。習慣開 n + 1 格，第 0 格不用。

【錯】n = 10^5 還用鄰接矩陣：10^{10} 格，記憶體直接爆掉。

2. BFS：佇列一層一層往外擴，第一次碰到就是最短步數

- BFS（廣度優先搜尋）用**佇列**：先進先出，離起點 1 步的點全部處理完，才輪到 2 步的點。
- 所以在「每條邊都算 1 步」的圖上，**第一次走到某個點時的步數就是最短步數**。
- dist 陣列一開始全設成 -1，兼當 visited：-1 代表還沒走到。

C++ BFS：每個點離起點幾步

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> dist(n + 1, -1);      // -1：還沒走到（兼 visited）
    queue<int> q;
    dist[1] = 0;
    q.push(1);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u])
            if (dist[v] == -1) {      // 第一次碰到就是最短
                dist[v] = dist[u] + 1;
                q.push(v);
            }
    }
    for (int u = 1; u <= n; u++) cout << dist[u] << " ";
    cout << "\n";
}
```

輸入	輸出
7 6 1 2 1 3 2 4 3 4 4 5 5 6	0 1 1 2 3 4 -1

點 7 沒有任何邊，走不到，所以保持 -1。

C++ 方格迷宮：最少走幾步

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    for (auto &row : g) cin >> row;
    vector<vector<int>> d(n, vector<int>(m, -1));
    int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
    queue<pair<int, int>> q;
    d[0][0] = 0;
    q.push({0, 0});
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        for (int k = 0; k < 4; k++) {      // 上下左右四個方向
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
            if (g[nr][nc] == '#' || d[nr][nc] != -1) continue;
            d[nr][nc] = d[r][c] + 1;
            q.push({nr, nc});
        }
    }
    cout << d[n - 1][m - 1] << "\n";
}
```

輸入	輸出
4 5 ..... .###. ...#. ##...	7

從左上角走到右下角，# 是牆。每一格就是一個點，上下左右相鄰就是邊。

陷阱提醒

【錯】從佇列拿出來才標記 visited：同一個點會被放進佇列好幾次，又慢又可能算錯。放進去的當下就標記。

【錯】方格題先檢查「有沒有出界」，再看 g[nr][nc]，順序反了會越界。

3. DFS：一路走到底再回頭，visited 是命根子

- DFS（深度優先搜尋）用**堆疊**（或遞迴）：一直往沒去過的鄰居走，走不下去才退回來。
- **數連通塊**：從 1 到 n 掃一遍，碰到還沒走過的點，答案加 1，再從它出發把整塊標記起來。
- **忘記 visited**：在有環的圖上會繞圈繞不完，遞迴到堆疊爆掉或超時。

C++ DFS（自己管理堆疊）：數連通塊

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<bool> vis(n + 1, false);
    int comps = 0, best = 0;
    for (int s = 1; s <= n; s++) {
        if (vis[s]) continue;
        comps++;                        // 發現一塊新的連通塊
        int cnt = 0;
        vector<int> st = {s};           // 用 vector 當堆疊
        vis[s] = true;
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            cnt++;
            for (int v : adj[u])
                if (!vis[v]) {
                    vis[v] = true;      // 放進堆疊時就標記
                    st.push_back(v);
                }
        }
        best = max(best, cnt);
    }
    cout << comps << " " << best << "\n";
}
```

輸入	輸出
8 5 1 2 2 3 3 1 4 5 6 7	4 3

四塊：{1, 2, 3}、{4, 5}、{6, 7}、{8}，最大的一塊有 3 個點。

C++ 遞迴版 DFS（短，但要小心深度）

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> adj;
vector<bool> vis;

void dfs(int u) {
    vis[u] = true;                    // 先標記，才不會繞圈
    cout << u << " ";
    for (int v : adj[u])
        if (!vis[v]) dfs(v);
}

int main() {
    int n, m;
    cin >> n >> m;
    adj.assign(n + 1, {});
    vis.assign(n + 1, false);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    dfs(1);
    cout << "\n";
}
```

輸入	輸出
5 4 1 2 1 3 2 4 3 5	1 2 4 3 5

遞迴太深怎麼辦？

- C++ 的遞迴深度大約幾萬到十幾萬層就可能「堆疊溢位」（看系統而定）。一條 10^5 個點的長鏈很危險。
- 最安全：點數到 10^5 的題目，一律用**自己管理的堆疊**或 **BFS**，不用遞迴。
- 只要「每個點都走到」就好的題目（數連通塊、標記），BFS 和 DFS 都可以，挑不會出事的那一個。

4. 樹：n 個點、n - 1 條邊、連通、沒有環

- **樹**：連通而且沒有環的圖；n 個點的樹剛好有 n - 1 條邊，任兩點之間只有一條路。
- **根**：指定一個點當最上面（題目常說「1 是根」）。根的鄰居是它的**小孩**，它是小孩的**父節點**。
- **深度**：離根幾條邊（根是 0）。**高度**：最大的深度。**葉子**：沒有小孩的點。
- **子樹大小**：自己加上所有子孫的點數，sz[u] = 1 + 所有小孩的 sz 加起來。

C++ 從根 BFS：深度與子樹大小

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) {      // 樹：n-1 條邊
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> par(n + 1, 0), dep(n + 1, 0), sz(n + 1, 1);
    vector<int> q = {1};                   // 根是 1；q 順便記 BFS 順序
    for (int h = 0; h < (int)q.size(); h++) {
        int u = q[h];
        for (int v : adj[u])
            if (v != par[u]) {             // 樹沒有環：不走回父節點
                par[v] = u;
                dep[v] = dep[u] + 1;
                q.push_back(v);
            }
    }
    for (int i = n - 1; i >= 1; i--)       // 倒著做：小孩一定先算完
        sz[par[q[i]]] += sz[q[i]];
    for (int u = 1; u <= n; u++) cout << dep[u] << " ";
    cout << "\n";
    for (int u = 1; u <= n; u++) cout << sz[u] << " ";
    cout << "\n";
}
```

輸入	輸出
7 1 2 1 3 2 4 2 5 3 6 5 7	0 1 1 2 2 2 3 7 4 2 1 2 1 1

第一行是每個點的深度，第二行是子樹大小。葉子是 4、6、7，高度是 3。

陷阱提醒

【錯】題目給的邊不一定是「父 子」的順序：1 5 和 5 1 都可能出現。一律當無向邊存，再從根 BFS 決定誰是父親。

【錯】用「度數 = 1」判斷葉子：根如果只有一個小孩，度數也是 1，會被誤算成葉子。

【對】BFS 的順序是「父親一定比小孩先出現」，所以倒過來做，就能由下往上算子樹大小，完全不用遞迴。

5. 二元樹的走訪：前序、中序、後序（識讀常考）

走訪	順序	記法
前序 preorder	根 → 左 → 右	「根」在最前面：一進來就印
中序 inorder	左 → 根 → 右	「根」在中間：左邊全部印完才印自己
後序 postorder	左 → 右 → 根	「根」在最後：兩個小孩都印完才印自己

C++ 二元樹的三種走訪

```cpp
#include <bits/stdc++.h>
using namespace std;

// L[u]、R[u]：u 的左、右小孩，0 代表沒有
int L[8] = {0, 2, 4, 5, 0, 0, 0, 0};
int R[8] = {0, 3, 0, 6, 0, 7, 0, 0};

void pre(int u) {                  // 前序：根、左、右
    if (u == 0) return;
    cout << u << " ";
    pre(L[u]);
    pre(R[u]);
}
void mid(int u) {                  // 中序：左、根、右
    if (u == 0) return;
    mid(L[u]);
    cout << u << " ";
    mid(R[u]);
}
void post(int u) {                 // 後序：左、右、根
    if (u == 0) return;
    post(L[u]);
    post(R[u]);
    cout << u << " ";
}

int main() {
    pre(1);  cout << "\n";
    mid(1);  cout << "\n";
    post(1); cout << "\n";
}
```

輸入	輸出
（不需要輸入）	1 2 4 3 5 7 6 4 2 1 5 7 3 6 4 2 7 5 6 3 1

這棵樹：1 的左右小孩是 2、3；2 只有左小孩 4；3 的左右小孩是 5、6；5 只有右小孩 7。

給前序 + 中序，還原整棵樹

- 前序的**第一個**一定是根。
- 在中序裡找到根：根的左邊全部是左子樹、右邊全部是右子樹。
- 數一數左子樹有幾個點，就能把前序切成「根 | 左子樹 | 右子樹」，兩邊再用同樣方法拆下去。

6. 樹直徑與樹上 DP 入門

- **樹直徑**：樹上距離最遠的兩個點之間的距離。
- 做法：從任一點 BFS 找到最遠點 a，再從 a BFS 找最遠點 b，a 到 b 的距離就是直徑。
- 邊有長度（權重）也一樣，只是距離要累加邊長；樹上任兩點只有一條路，所以BFS照樣算得出距離。
- **樹上 DP**：每個點的答案由小孩的答案組合出來（像子樹大小）。用 BFS 順序倒著做，就是「由下往上」。

C++ 兩次 BFS 求樹直徑

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
vector<vector<int>> adj;

vector<int> bfs(int s) {
    vector<int> d(n + 1, -1);
    queue<int> q;
    d[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u])
            if (d[v] == -1) { d[v] = d[u] + 1; q.push(v); }
    }
    return d;
}

int main() {
    cin >> n;
    adj.assign(n + 1, {});
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> d1 = bfs(1);
    int a = max_element(d1.begin(), d1.end()) - d1.begin();
    vector<int> d2 = bfs(a);              // 從最遠點 a 再做一次
    cout << *max_element(d2.begin(), d2.end()) << "\n";
}
```

輸入	輸出
7 1 2 1 3 2 4 2 5 3 6 5 7	5

最遠的兩點是 7 和 6：7 → 5 → 2 → 1 → 3 → 6，共 5 條邊。

7. 拓撲排序入門（選修）

「先修完 A 才能修 B」這種有方向的關係，畫成有向邊 A → B。拓撲排序就是排出一個做事順序，讓每條邊的起點都排在終點前面。做法（Kahn 演算法）：先把「入度 = 0」（沒有人要先做）的點放進佇列，每拿出一個點，就把它指向的點入度減 1，減到 0 就放進佇列。

C++ 拓撲排序（Kahn 演算法）

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    vector<int> indeg(n + 1, 0);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;                // u 要排在 v 前面
        adj[u].push_back(v);          // 有向邊：只加一次
        indeg[v]++;
    }
    vector<int> q;
    for (int u = 1; u <= n; u++)
        if (indeg[u] == 0) q.push_back(u);
    for (int h = 0; h < (int)q.size(); h++)
        for (int v : adj[q[h]])
            if (--indeg[v] == 0) q.push_back(v);
    if ((int)q.size() < n) cout << "cycle\n";   // 有環就排不完
    else {
        for (int u : q) cout << u << " ";
        cout << "\n";
    }
}
```

輸入	輸出
5 5 1 2 1 3 3 2 2 4 5 4	1 5 3 2 4

排出來的點數少於 n，代表圖上有環（互相等對方先做），不可能排出順序。

讀完這份，你應該要會

- □ 會用 vector<vector<int>> adj(n + 1) 建鄰接串列，無向圖加兩次邊
- □ 會用 BFS 求最短步數（包含方格迷宮），知道「放進佇列時就標記」
- □ 會用堆疊做 DFS 數連通塊，知道忘記 visited 會繞圈
- □ 點數到 10^5 時，知道遞迴可能太深，會改用堆疊或 BFS
- □ 會從根 BFS 算出深度、葉子、子樹大小
- □ 看到二元樹，能寫出前序、中序、後序
- □ 知道兩次 BFS 求樹直徑的做法

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室

IG @jf_chuang

LINE 諮詢 jenfoo2021